FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

TCG6223 COMPUTER GRAPHICS

PROJECT REPORT

Trimester March/April 2025 (Term 2510)

Team Name: Affectionate

Team members

No.	Name	ID	Lab Section	Program
1	KANG XUAN XIN	241UT240W1	1F	AI
2	CHEONG KAI JIN	241UT240K7	1F	AI
3	CHONG ZHENG CHENG	241UT240LQ	1F	AI
4	KAY HEAN CUN	241UT240KF	1F	AI

TABLE OF CONTENTS

INTRODUCTION 6
Modeling Pipeline 6
Animation Pipeline 6
Our Character Models 7
Voltorb 7
Doraemon 8
Chicken Jockey 8
DOCUMENTATIONS OF VIRTUAL MODEL 10
Virtual Model 1 – Pokémon (Voltorb) 10
Voltorb's Body Sphere 10
Voltorb's White Eyes 11
Voltorb's Black Pupils 12
Final Virtual Model for Voltorb 13
Textures 13
Animations for Voltorb 13
Idle Animation 13
Attack Animation 14
Defend Animation 15
Regeneration Animation 16
Victory Animation 16
Defeat Animation 17
Virtual Model 2 – Doraemon 18
Doraemon's Head 18
Doraemon's Torso 18
Doraemon's Arm 19
Doraemon's Leg 19

Doraemon's Tail 20
Doraemon's Textures 20
Head Texture 20
Body Texture 21
Arm Texture 21
Leg Texture 22
Tail Texture 22
Final output model 23
Items 23
Air gun 23
Shield 24
Dorayaki 24
Animations for Doraemon 27
Idle Animation 27
Attack Animation 27
Defend Animation 28
Regeneration animation 28
Victory animation 29
Defeat animation 29
Virtual Model 3 – Chicken Jockey 31
Chicken's Feet 31
Chicken's Torso 31
Chicken's Wings 32
Chicken's Head 32
Baby Zombie's Legs and Arms 33
Baby Zombie's Body 33
Baby Zombie's Head 34

Final Output Model 34
Items 35
Cobblestone Block 35
Textures 36
Chicken Leg, Wing, Torso and Head 36
Baby zombie Leg, Arm, Torso and Head 36
Cobblestone, Cobblestone shard, Egg, Egg shard, Potion, Effect 1-8 36
Animations for Chicken Jockey 37
Idle Animation 37
Attack Animation 37
Defend Animation 38
Regeneration Animation 38
Victory Animation 39
Defeat Animation 39
Virtual Environment 40
The Skybox 40
Court Model 40
Confetti 42
Graphical User Interface (GUI) 42
Menu Screen 43
Battle Screen 43
Result Screen 44
USER MANUAL AND INSTRUCTIONS 45
General Gameplayh 45
Menu Screen 45
Battle Screen 46
Result Screen 46

Other Controls 46
SCREENSHOTS AND SHOWCASES 48
Console at Startup 48
Menu Screen 48
Initially 48
Changed Characters 49
Battle Screen 49
Battle started 49
Characters fighting 50
Result Screen 50
Player 1 wins 50
Others 51
DEMONSTRATION VIDEO LINK 51

INTRODUCTION

Modeling Pipeline

Our group has two modelling workflows: **"Blender model > export to .obj file > convert .obj to .txt file > read .txt file into program"** and **"Build model using OpenGL code > compile and view the model"**. We mostly use Blender to create our model because it makes it easier to create models without constantly changing parameters in the code and recompiling to view the results. It also allows us to create much more complex 3D models in a shorter amount of time. But we still have some models and elements built entirely using OpenGL code.

Our character models are all original and created by each member of the team. Blender is used ONLY for generating vertices and UV mapping information (textures), which alligns with the provided guidelines.

Animation Pipeline

Our team uses a "delta time" system to control animations. In a simpler animation system, the program usually uses each drawing cycle (frame update) to update the animations and movements by a constant scale, this results in inconsistent animation speed when the framerate is changing. In a "delta time" system, frames are updated on a fixed interval and the animations are updated with "delta time" (the duration between current frame and previous frame) scale to ensure consistent animation in various framerates. This also allows us to control the framerate.

The animations of our models are hard-coded (defined in the code instead of data-driven) and are hand-tuned by members of our team. All the animations are updated based on the "delta time" system our team uses.

To animated limbs and body parts, each model is separated into multiple model parts like head, torso and limbs. Then, we anchor the parts into their corresponding position and apply transformations depending on the animation playing.

Our program also introduces particles, more specifically a particle drawing technique called "Billboarding" which allows our program to have flat 2D sprite particles in a 3D environment.

Our Character Models

We have chosen three characters from Doraemon, Pokémon and Minecraft as our models. Here are some images of their blind box collections:

Introducing our chosen blind box characters:

Voltorb

Voltorb is a circular Pokémon, with red colour on the upper body and grey-white colour on the lower body, resembling a Poké Ball. It has angry-looking eyes, and it has no mouth. It is an Electric Pokémon and can explode if it gets threatened.

The left figure shows a reference/sample image of the Pokémon (Voltorb).

There are a total of six animations for the voltorb:

Idle - The voltorb tilts its body around its position.

Attack - The voltorb rolls out at a high speed towards its opponent, hits them, and comes back.

Defend – The voltorb spins in a high speed to reflect attacks.

Regenerate – The voltorb absorbs energy(electricity) from the surroundings.

Victory – The voltorb gives a cool look.

Defeat – The voltorb shakes its head.

Doraemon

Doraemon is a robotic cat from future world in Japanese anime and manga. The purpose of Doraemon arrival is to help Nobita to study well. His body is mostly blue, but the face, hand, belly and feet are white. The characteristic of Doraemon is the magical pocket on his belly that stores many futuristic gadgets. Dorayaki is the favorite food for Doraemon.

The left figure shows a reference/sample image of Doraemon

There are a total of six animations for Doraemon:

Idle - Doraemon shakes his body up and down

Attack - Doraemon takes out his air gun and raises it to shoot

Defend - Doraemon call a shield to defend

Regenerate - Doraemon eats dorayaki to recover mana

Victory - Doraemon raise up hands and jump up

Defeat - Doraemon lowers his head and shakes it

Chicken Jockey

Chicken Jockey is a mob from Minecraft. It consists of two major parts: the chicken and the baby zombie sitting on top. The chicken has yellow feet, orange beaks and white feathers, while the zombie has green skin and wears blue clothing. They have a pixelated appearance which is an artistic style from the game Minecraft.

The left figure shows a reference/sample image of the Chicken Jockey mob.

There are a total of six animations for Chicken Jockey:

Idle – The chicken flaps its wings. The chicken's body, head and zombie's arm will bob slowly.

Attack – The baby zombie holds an egg in his hands and throws it towards the opponent.

Defend – The baby zombie builds a wall with cobblestone and breaks it after.

Regenerate – The baby zombie throws a splash potion of regeneration on himself.

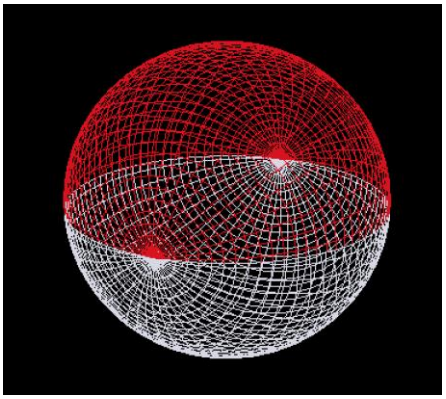
Victory – The baby zombie does his little dance.

Defeat – The baby zombie lowers his head with disbelief.

DOCUMENTATIONS OF VIRTUAL MODEL

Virtual Model 1 – Pokémon (Voltorb)

Voltorb's Body Sphere



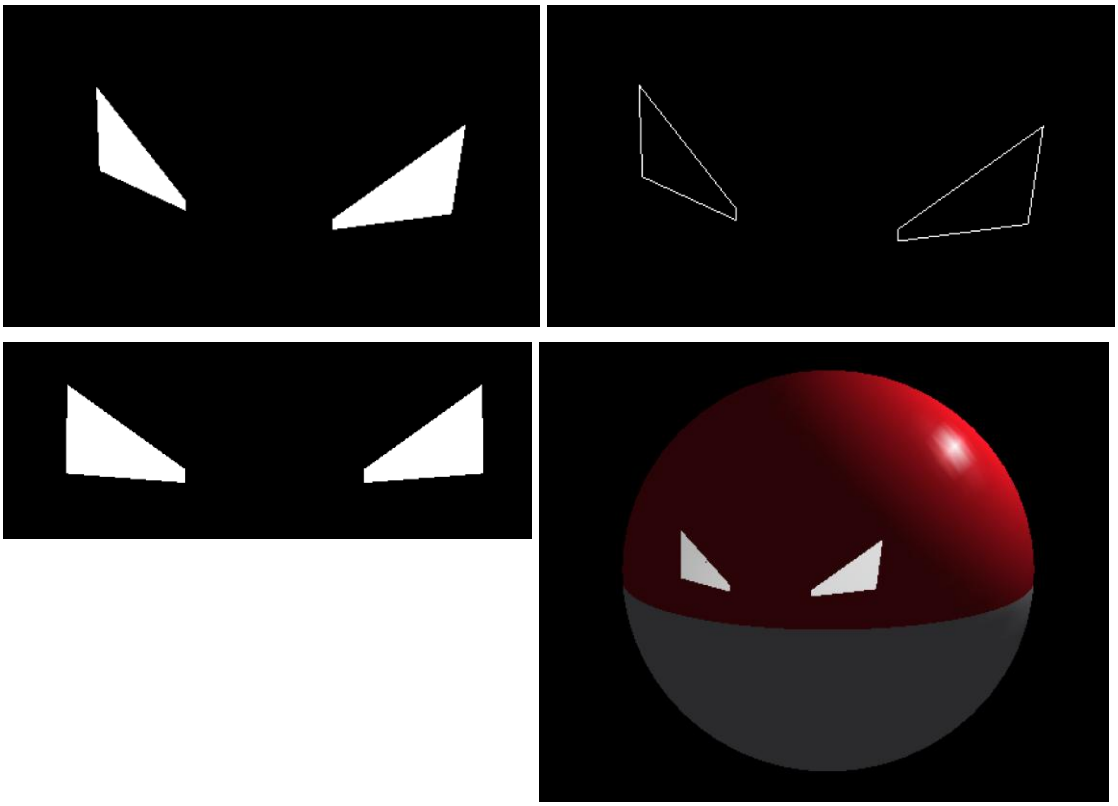
The figure above shows the body sphere for the Voltorb.

Firstly, we begin creating the Voltorb model by first drawing a sphere. The body sphere of Voltorb is rendered by combining a clipped lower light gray half and a clipped upper red half. This can be accomplished with the gluSphere() function in OpenGL, but with the top and bottom half split with clipping planes.

The bottom half is drawn first. Before it's rendered, a clipping plane is established such that only the lower half of the sphere (where Y < 0) will be visible. This half is rendered in light grey with glColor3f (0.85f, 0.85f, 0.9f). glClipPlane() takes care of the clipping and this mode of operation is enabled temporarily for the duration of the draw.

Then the top half is rendered. This time, the top half (Y > 0) is displayed using a different clipping plane. Colouring is switched to red with glColor3f (0.85f, 0.08f, 0.13f) to match the top color of the signature of Voltorb.

Voltorb's White Eyes



Figures above show somes of the views for Voltorb's White Eyes.

Now, we need to draw the White eyes of the Voltorb. Since Voltorb's surface is curved like a sphere, the eyes must smoothly conform to it. So, rather than utilising a constant z-coordinate, we have to compute the Z location for every vertex in such a manner that it fits on the surface of the sphere exactly.

This is done by the sphere equation: $z = \sqrt{r^2 - x^2 - y^2}$

Where r is the radius, and (x,y) are the chosen coordinates. To make the white eyes appear above the body sphere (to prevent z-fighting), we need to move it a little further by adding a small offset for the Z coordinate (e.g., +0.135f). After calculating the Z coordinate for each vertex, we can begin to draw the eyes using GL_QUADS as the eye contains four vertices. The colour used will be white, specified using glColor3f(1, 1, 1).

Voltorb's Black Pupils

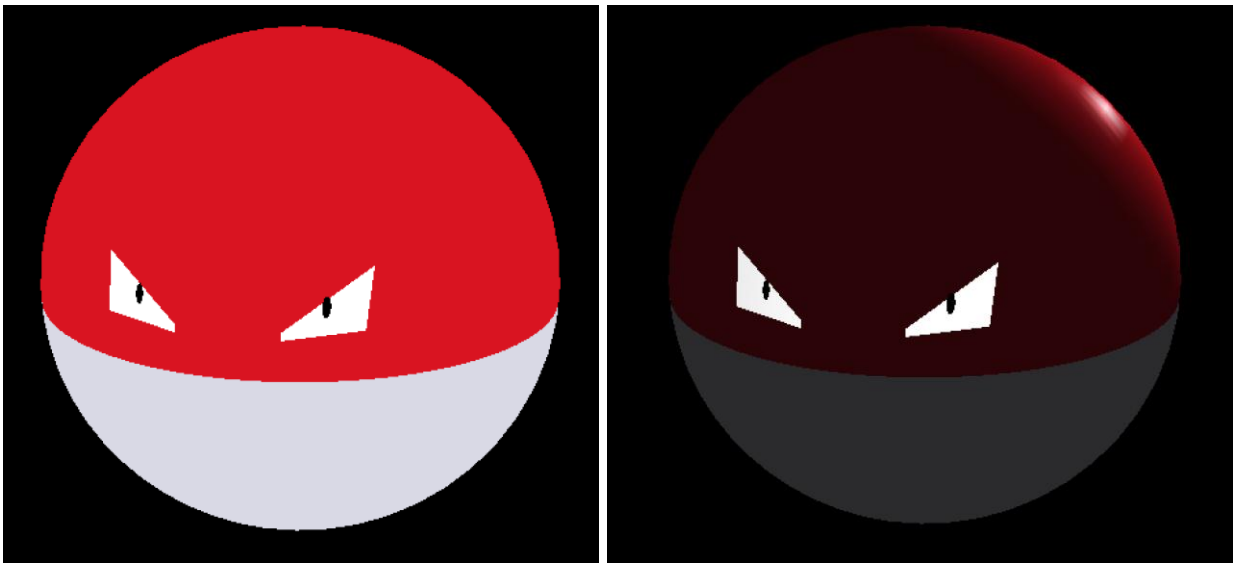
The figure above shows the White Eyes and the Black Pupils of Voltorb.

To draw the black pupils on Voltorb, we use a loop to build a flat oval shape that sticks nicely on top of the curved sphere surface. In the code, we first set the drawing colour to black using glColor3f(0, 0, 0). Then, we choose a center point for each pupil, one on the left side (plx, ply) and one on the right (prx, pry). These positions are placed near the top center of each white eye shape.

To draw the pupils of the black color for the eyes of Voltorb, we use glBegin(GL_POLYGON) to inform OpenGL we are going to draw a filled shape by joining a number of points and providing a smooth line between the last and the first ones. An ellipse, a stretched circle, in our case, is what we are going to draw. To draw this shape, we have 36 small points around the center of the ellipse, and we place them evenly in a circular path using angles ranging from 0 to 360°. To determine the location of the points, we use two mathematic functions: cos(angle) and sin(angle). Using the cos function, we get the x-position (left to right), and the sin function lets us know the y-position (top to bottom) for a given angle. Using the example, 0°, the cos(0) will return 1, and the sin(0) will return 0, and the point will be in the far-right position. 90°, cos(90°) returns 0 and sin(90°) returns 1, and the point will be in the top position. This goes on in a circular manner. To make the shape an oval instead of a full circle, we need to scale the x and the y values by multiplying them with x_radius and the y_radius, meaning the ellipse will be wider or taller based on the values used for them.

Once we have the x and y locations for the ellipse point, we wish to compute z such that the pupil will curve on the surface of the sphere. We do this with the mathematical formula for a sphere's surface again that we used just now. This puts the pupil on the curved surface of the head of Voltorb. Then we add a small adjustment to the z value such that the black pupil will be slightly above the surface of the red, avoiding flicker or overlap due to z-fighting.

Final Virtual Model for Voltorb

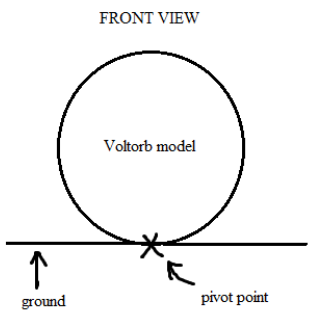


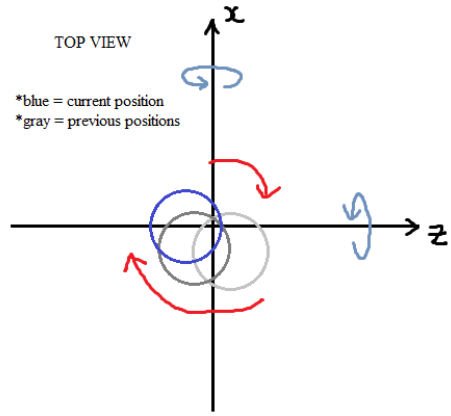
Textures

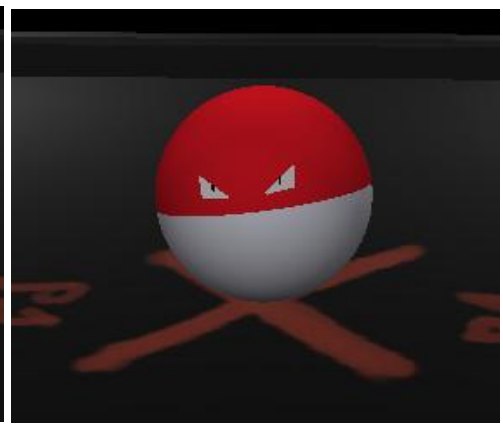
These are textures used for the Voltorb's animation

Animations for Voltorb

Idle Animation

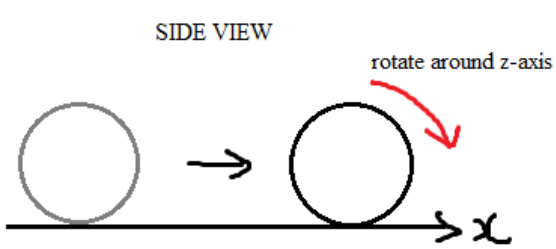


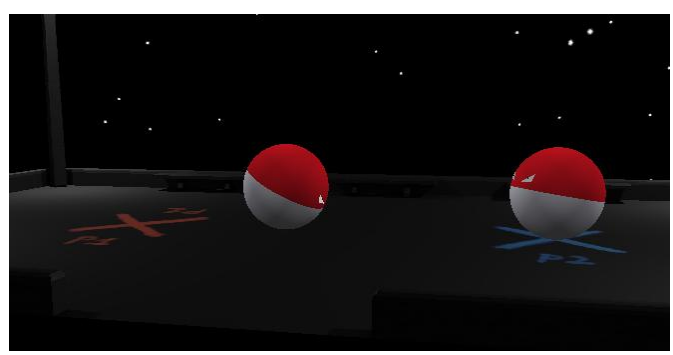




The idle animation for Voltorb is done by rotating its body around the pivot set right under the model, approximately where it touches the ground. The body is then rotated around the X and Z axis to make it look like it's tilting around (like how a tilting doll/ottogi/roly-poly/tumbler toy would). Rotation degrees are controlled by a sin function with time: rotx = sin(animtime*1.2) * 15

Attack Animation

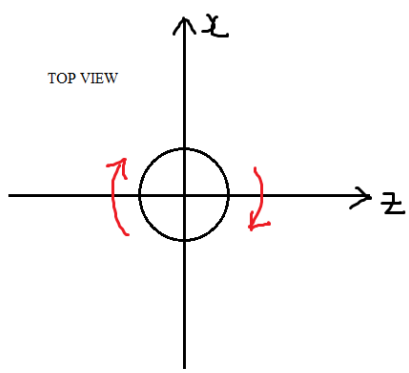


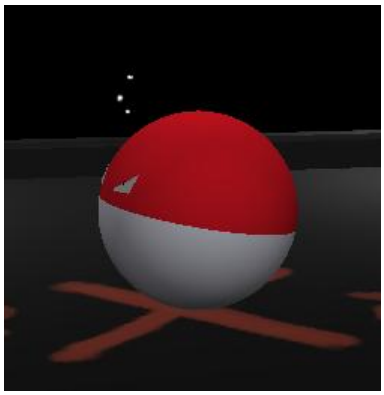


The attack animation is a rolling motion towards the opponent. This is achieved by increasing the X position (forward) of the body and rotating the body around the Z-axis. After the body has

reached the opponent, it will then decrease the X position, which makes the body go back to its original position. When it reaches its original position, it will play the idle animation.

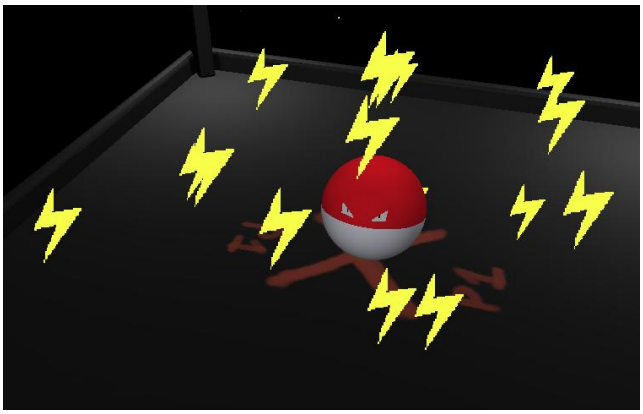
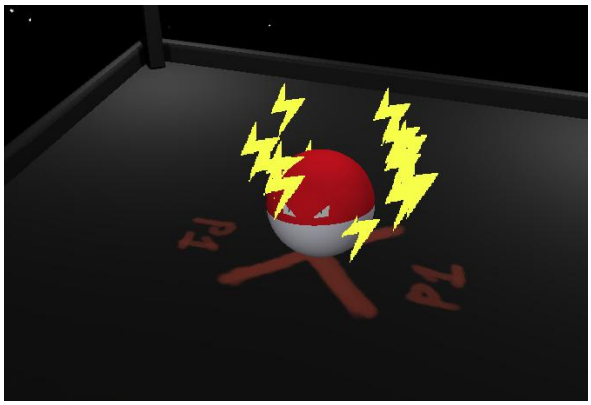
Defend Animation





The defend animation is a fast-spinning motion. The body spins around its origin. The body is also slightly rotated upward to make the spinning motion more visible. It spins for 3 seconds before switching back to the idle animation.

Regeneration Animation

The Voltorb will absorb “energy” from its surroundings. This is done by instantiating particle objects (using a class called Energy in the code), giving them a random starting position ranging from -3 to 3 around the body in all axes, then slowly move them towards the body’s origin. The movement uses a constant linear interpolation function which makes the particle have a “ease out” motion: pos = lerp(pos, 0, deltatime*speed).

Victory Animation

Voltorb look towards the camera and give a “cool” look, the yellow shine near the eye is a particle. The particle will bob up and down. It also changes its size slightly. The bobbing and size change is controlled by sin functions. The eye appears darker here is because of the lighting, not a glitch.

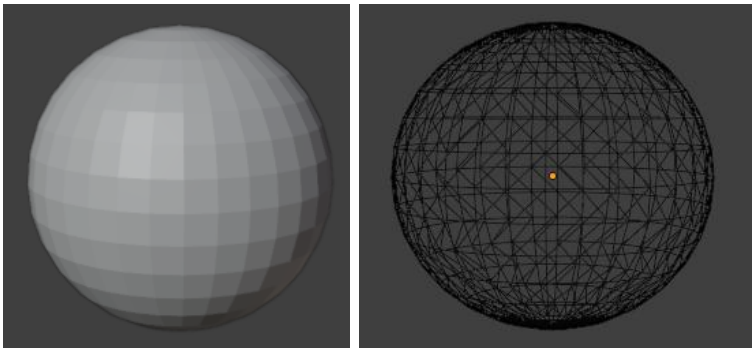
Defeat Animation

Voltorb looks 40 degrees downwards and shakes its head, the head shaking animation is controlled by sin function: rot = sin(animtime*1.6)*20.

<u>Virtual Model 2 – Doraemon</u>

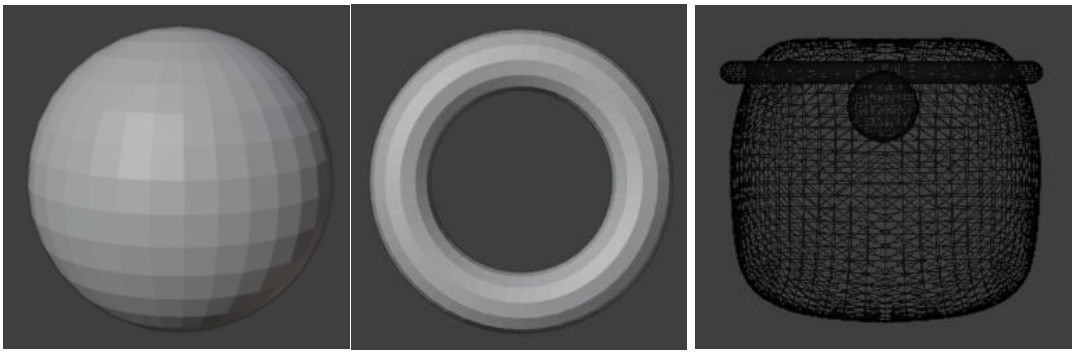
The Doraemon's model and texture is made with Blender, we first build the model in Blender and generate the vertices information to import into the GLUT program.

Doraemon's Head



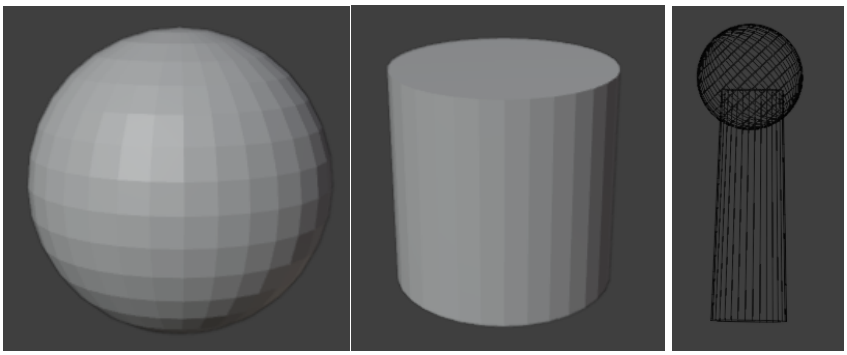
This is Doraemon's head, we just use the sphere shape. Then, adjust the size. The XYZ scale of the head are 1.494.

Doraemon's Torso



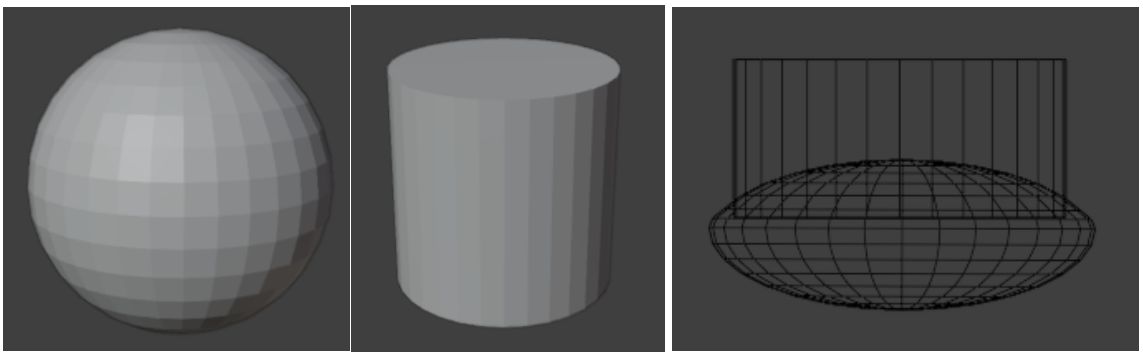
Firstly, we create Doraemon's bell model by using sphere. After creating the shape, we adjust the size to make it smaller. Secondly, we create a ring(Torus) shape as Doraemon's collar and squeeze and shrink it to get the suitable size. Thirdly, we create one more sphere as Doraemon's body. The body shape will be adjusted by modifying the vertices one by one to get the shape that looks like a square and a circle. Finaly, we get complete body components by joining these three components.

Doraemon's Arm



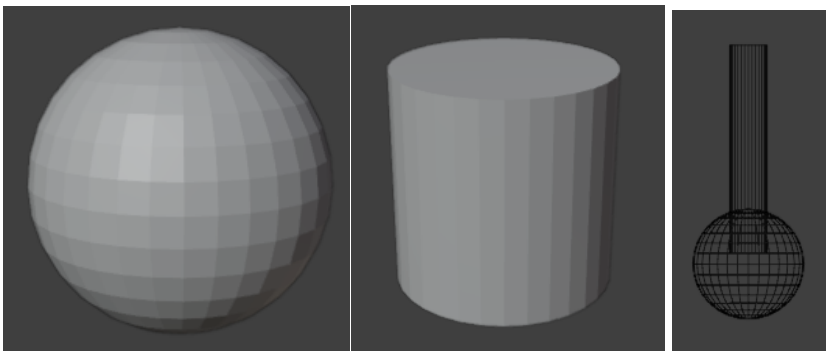
This is Doraemon's arm components, we use the combination of sphere and cylinder shape. The sphere shape only needs to adjust the size, that is used to represent hand. However, the cylinder shape needs to be adjusted to have one end larger than the other, so that the arm will look more natural. Then, we combine these two components to get a wholly arm component.

Doraemon's Leg



This is Doraemon's leg components, we also use the combination of sphere and cylinder shape which is same as arm components. The sphere that looks like UFO which is after squeeze and shrink used to represent sole. But the cylinder shape only needs to adjust its width and height to make it short and thick. Then we combine it and get the model of Doraemon's short legs components.

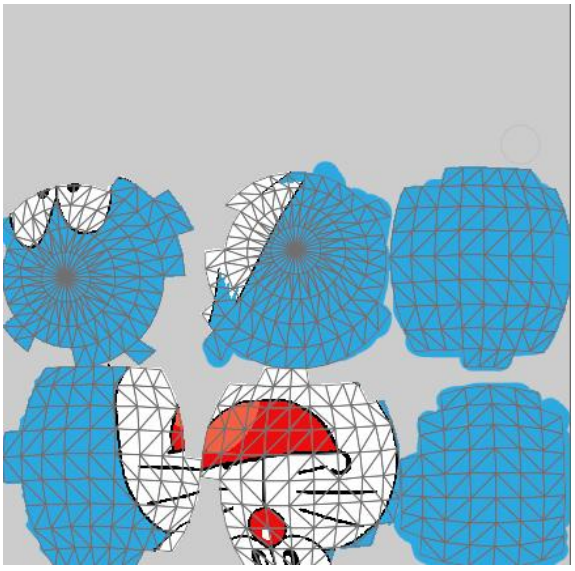
Doraemon's Tail



This is Doraemon's tail component, we also use sphere and cylinder shape to create it. The sphere is just adjusting the size to be small while the cylinder shape needs to adjust its width and height to shape into thin strips.

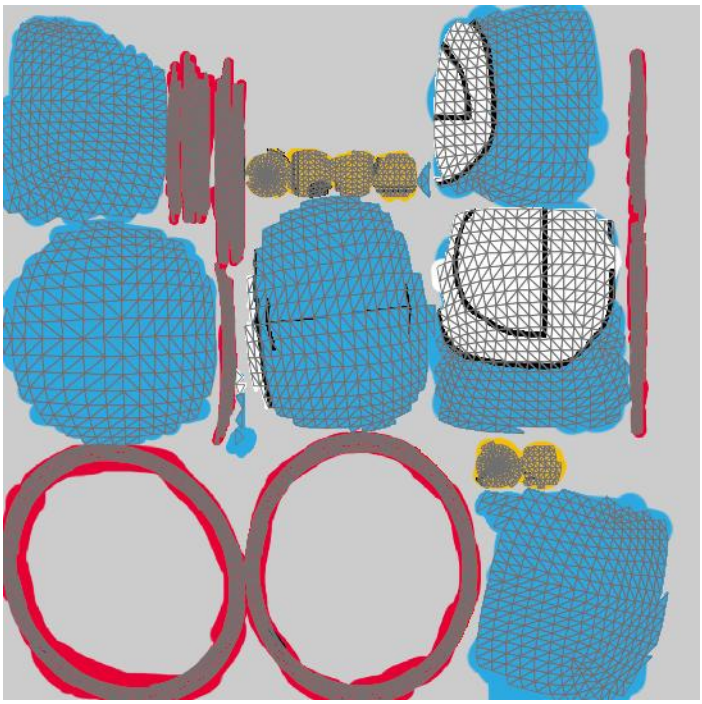
Doraemon's Textures

Head Texture



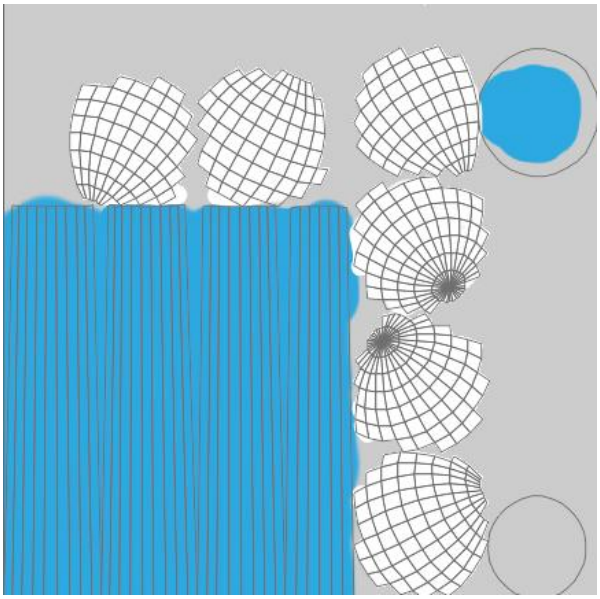
The colours that used for head components are Blue(#2CA9E1), White(#FFFFFF), Black(#000000) and Red(#EB0E0E, #E60033, #F56042).

Body Texture



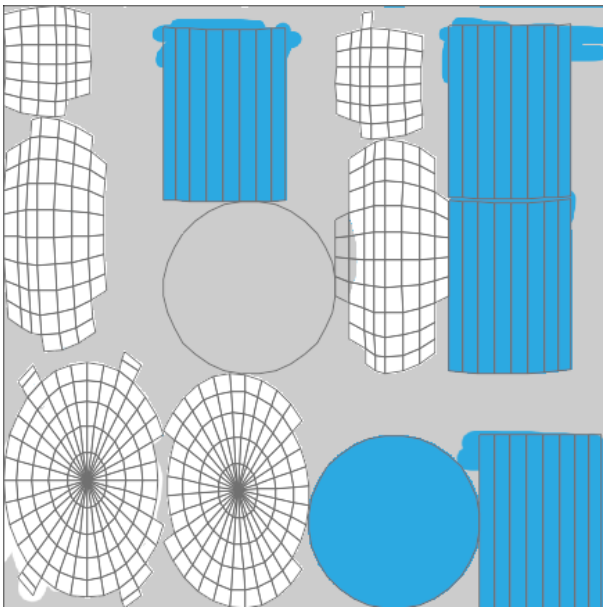
The colors that are used for body components are Blue(#2CA9E1), White(#FFFFFF), Black(#000000), Red(#E60033) and Yellow(#F6C000).

Arm Texture



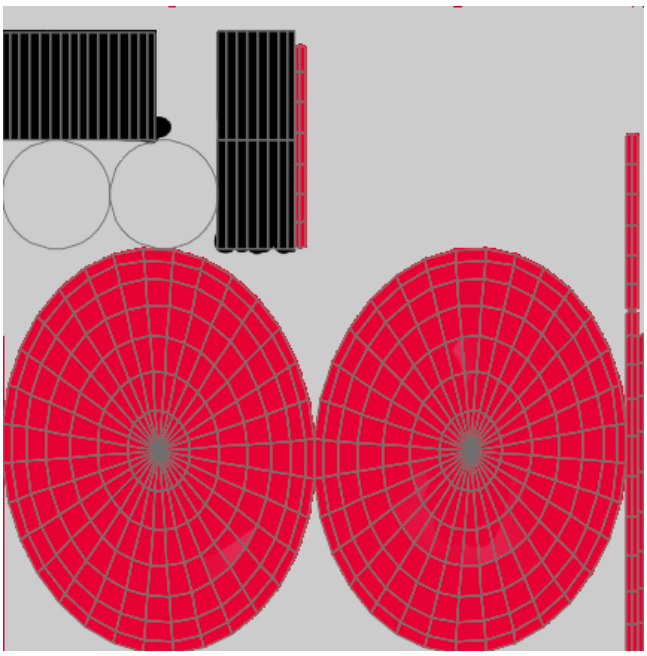
The colors that are used for arm components are Blue(#2CA9E1) and White(#FFFFFF).

Leg Texture



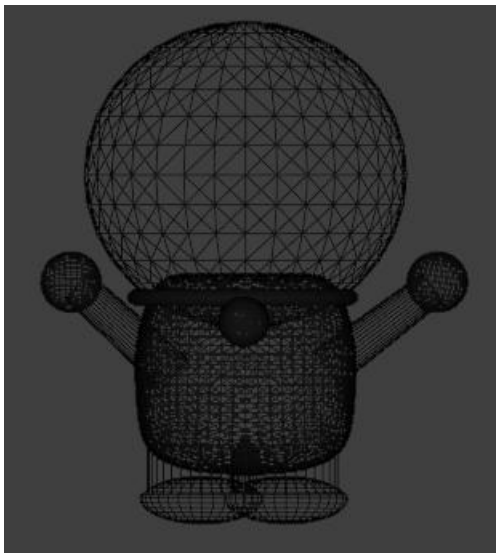
The color that is used for leg components are Blue(#2CA9E1) and White(#FFFFFF).

Tail Texture



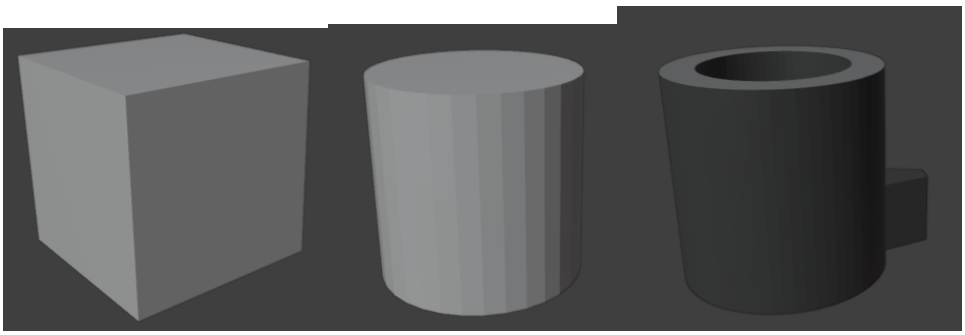
The colors that are used for tail components are Black(#000000) and Red(#E60033).

Final output model



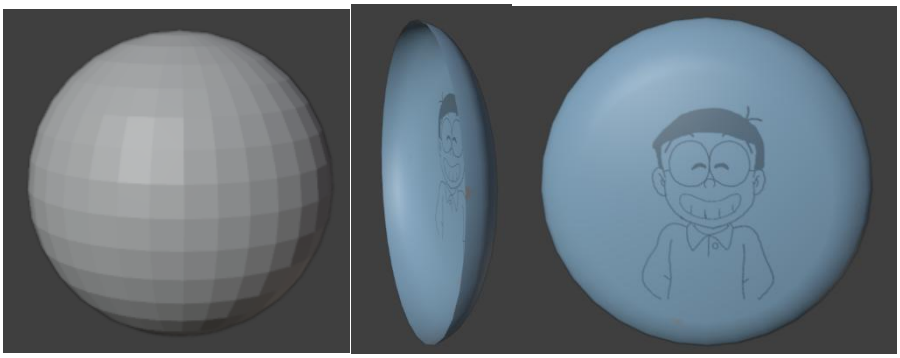
Items

Air gun



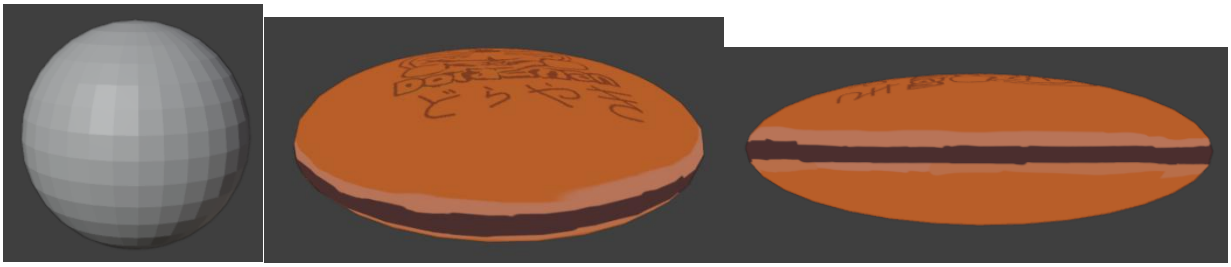
This is air gun model, we create the gun's body by using 2 cylinders that one big and one small. We put the small inside the bigger one and merge them to get a hollow cylinder. After this, we adjust four upper point of the cube to make it shape like a scope. Finaly, we combine it to get the air gun model.

Shield



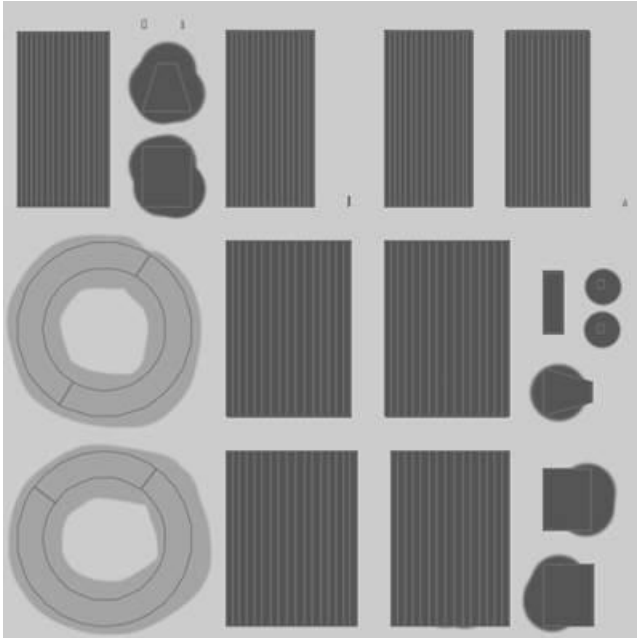
This is shield model, we create it by using a sphere shape. We directly remove close to three quarters of the sphere to get a shape look like shield.

Dorayaki

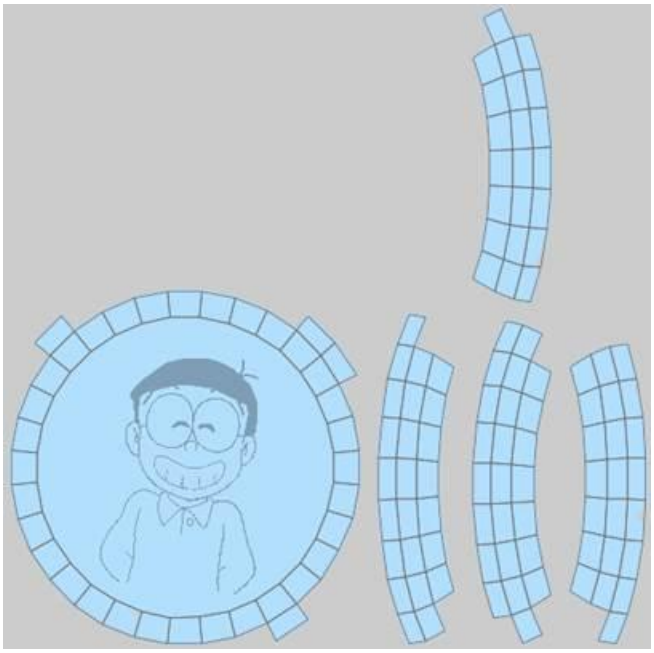


This is dorayaki model, we create it by using a sphere shape. We just squeeze up and down the sphere to form a shape that looks like a UFO.

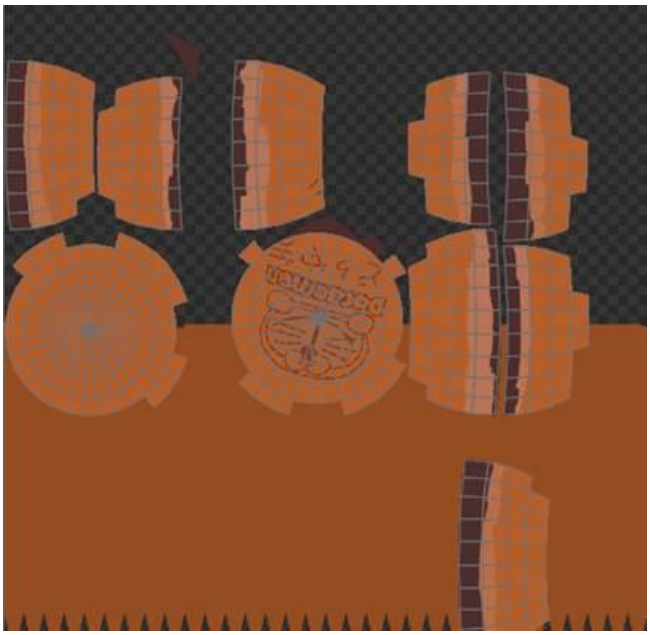
This is the texture for the skill animation



The colors that are used for air gun is Gray(#555555 and #A4A4A4). The bullet uses a Png image to represent it.



The colors that are used for shield are Blue(#B3DFFF) and Gray(#7F9EB5)



The colors that are used for food(dorayaki) are Orange(#B85E29) and Chocolate(#4B2E2E, #803D21)

Animations for Doraemon

Idle Animation

For the idle animation, which is Doraemon's body shaking up and down, we use a mathematical equation to achieve it.

bodybob = sin(animtime * 3.8) * 0.065

3.8 creates smooth wave speed and 0.065 represents the height of the motion shaking. Then we put bodybob inside glTranslatef and the body will slight up and down over time.

Attack Animation

The raise right hand animation was created using linear interpolation of the armt from 0 to 90 degrees then let the arm be smoothly rotated on Z-axis. This code is put inside the right arm's drawing code. While the hand is rising up, the air gun is drawn on the hand. After 0.5 seconds of arm moving up, bullets appear and move from position 2 to 8 along X-aixs. The bullet movement also uses linear interpolation for smoothness, and then the bullet disappears after 2.5 seconds.

Defend Animation

Sheildt is used to smoothly animation of movement and scaling the shield. Since we make the shield have diagonal movement and the scale from small to big, we use linear interpolation of the X-axis, Y-axis and Scale inside function drawbody(). When it moves on X-axis and Y-axis at the same time, we can have a movement that looks like diagonal movement. The result is the shield will appear from small to big when it moves forward and up. It will disappear in 3 seconds.

Regeneration animation

The animation of skill 3 is a little bit the same as skill 2, the difference is we set it to move from front to back and bottom to top. The food dorayaki appears and moves back from position 2 to 0.2 on X-axis, and the scale reduces from 1 to 0.3. At the same time, both hands will rise and take action like eating food. We use the same code for both hands which is we rotated from 0 to 135 degrees by using linear interpolation on Z-axis. Then, the dorayaki will move inside the mouth and disappear after 1 second.

Victory animation

For the victory animation, Doraemon will raise both hands and jump when he wins the game. For the hand raising we call back the animation of skill 3 and add one more condition which is playinganim == 5 that represent victory state. But the armt inside this animation is using code below to allow it to move repeatedly and smoothly.

armt = fabs(sin(animtime * 3));

fabs(absolute value) are used to make sure the value is always positive.

Then, we use another sin function to make Doraemon jump up and down movement on Y-axis.

float jumpy = fabs(sin(animtime * 3));

glTranslatef(0, jumpy, 0);

Defeat animation

For the defeated animation, Doraemon's head shakes side to show sadness and defeat. This animation is created by the combination of translation and rotation. Firstly, we translate the head to make it move backward a little bit to make the face not blocked by the body. Secondly, we

rotate its head -30 degrees downwards around the Z-axis. Then, we create a sin function to let the head can smoothly between -45 and 45 degrees.

```
float headdeg = sin(animtime * 2) * 45;
```

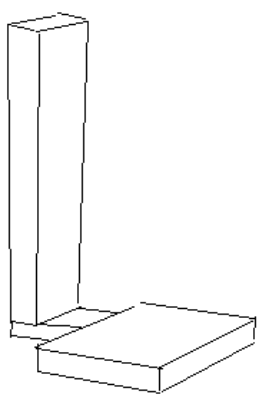
2 represents the speed and 45 represents positive and negative degrees.

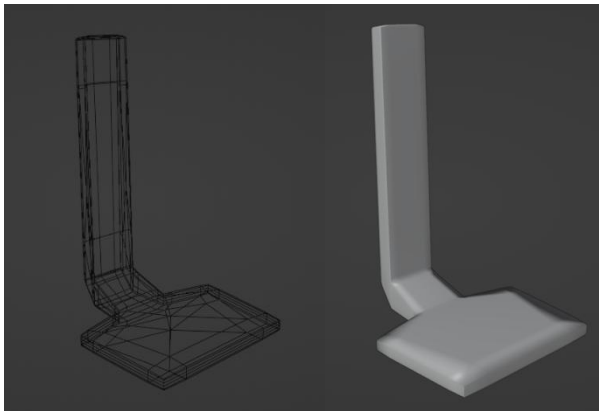
Virtual Model 3 – Chicken Jockey

The chicken jockey model and texture are created in blender, then the vertices information is generated in OBJ format and imported to the program.

We went for a design with smoothed edges rather than the flat look in the original game, so it has a toy-like appearance to it. It also looks better with lighting.

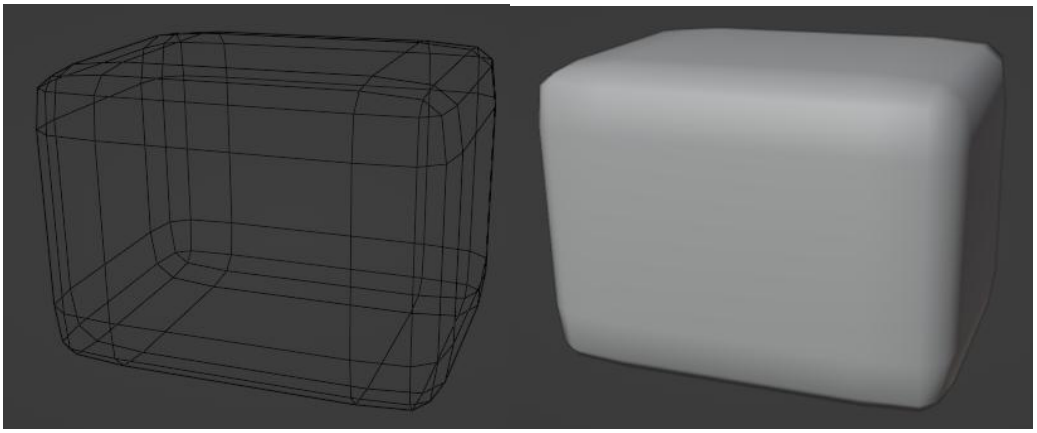
Chicken's Feet





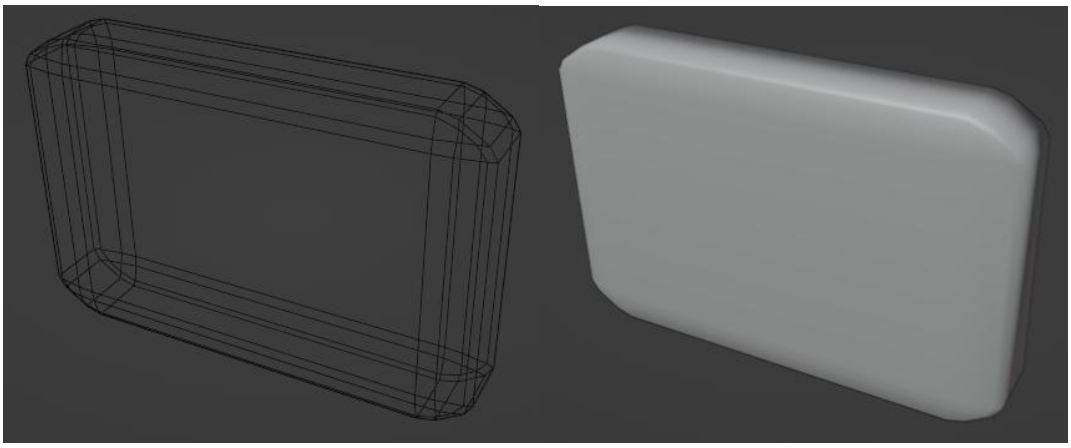
The chicken's feet are first made with three rectangle cubes that are connected, then the rectangle cubes are beveled into a smoother shape. The left foot and right foot use the same model.

Chicken's Torso



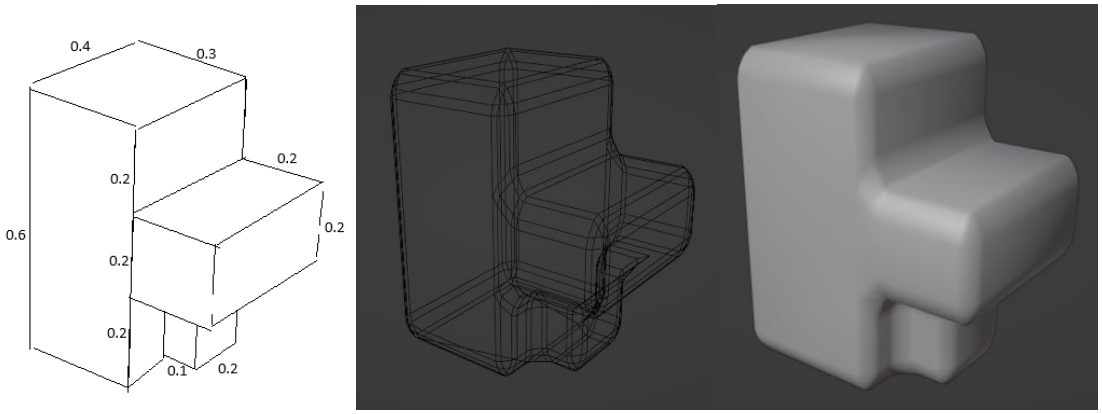
The chicken's torso is a simple rectangle cube with dimensions of {1.6, 1.2, 1.2}. The edges are then smoothed.

Chicken's Wings



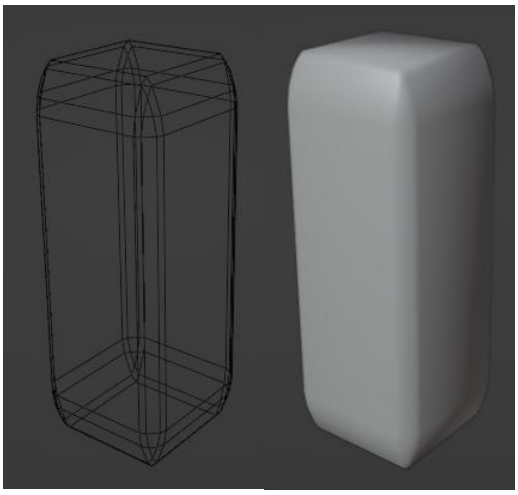
The chicken wing is a flatter simple rectangle cube with dimensions of {1.2, 0.2, 0.8}. The left wing and right wing use the same model.

Chicken's Head



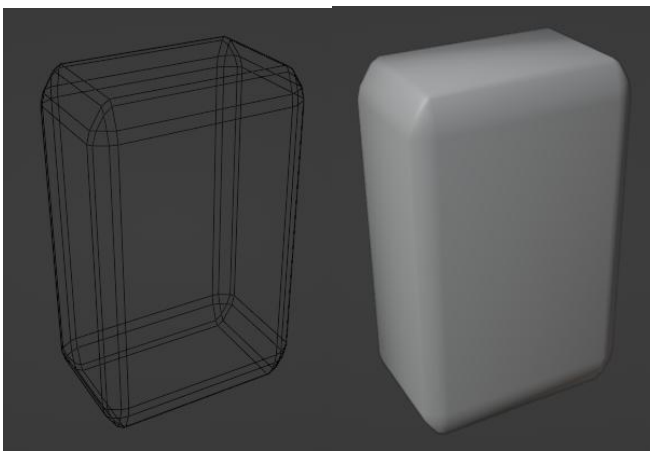
The chicken's head is made with three rectangle cubes connected, one resembles the main head part, another resembles the beak and the last resembles the wattle (red thing under the chicken's beak). The edges are then smoothed.

Baby Zombie's Legs and Arms



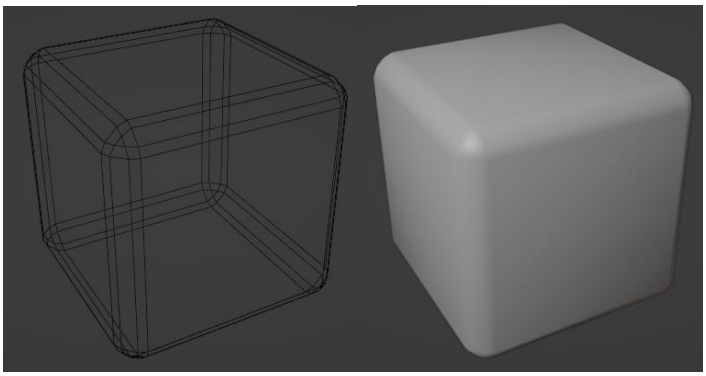
The baby zombie's leg and arm are made using a rectangle cube. The edges are then smoothed. The left and right leg and arm use the same model.

Baby Zombie's Body



The body is a rectangular cube with dimensions {0.6, 1, 1.6}. The edges are then smoothed.

Baby Zombie's Head

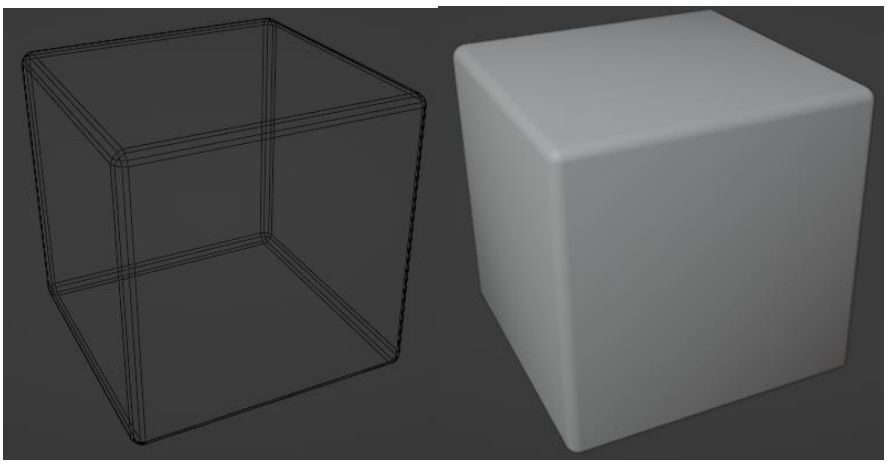


The head is a cube with dimensions {1.3, 1.3, 1.3}. The edges are then smoothed.

Final Output Model

Items

Cobblestone Block

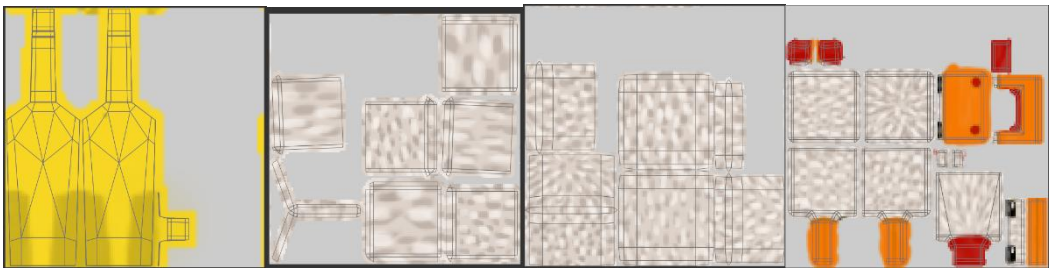


The cobblestone block model is used when playing the defend animation. It is a cube with dimensions {1, 1, 1}. The edges are also then smoothed to give it a toy-like look, matching the other models.

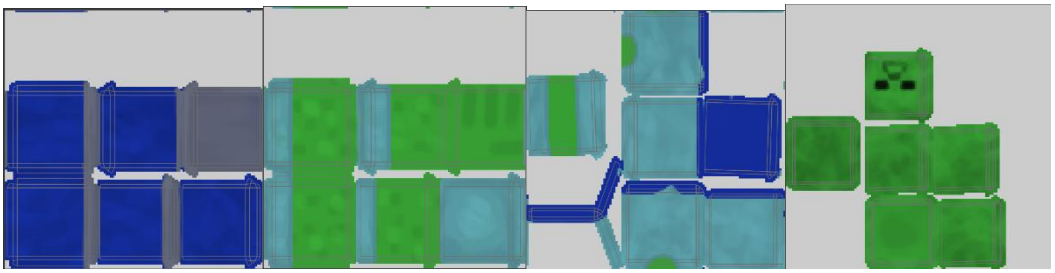
Textures

These are the textures used by the models. They are mapped to their corresponding vertices.

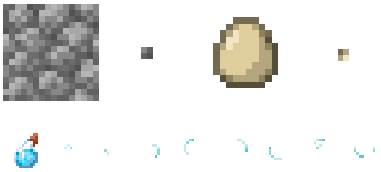
Chicken Leg, Wing, Torso and Head



Baby zombie Leg, Arm, Torso and Head



Cobblestone, Cobblestone shard, Egg, Egg shard, Potion, Effect 1-8



The cobblestone, egg and potion textures are taken from Minecraft.

Animations for Chicken Jockey

Idle Animation

The chicken's head and body will bob up and down, their Y-offsets are both controlled by a sin function. The head's Y-offset sin function has a slight offset in time, so that it appears livelier. The chicken's wings are always flapping, controlled by a sin function, the rotation degrees are limited to 0-90 degrees. The chicken animation is the same across all animations.

The baby zombie's body moves with the chicken body, the arms of the baby zombie rotate slightly in the X and Z axis, so it has subtle movements.

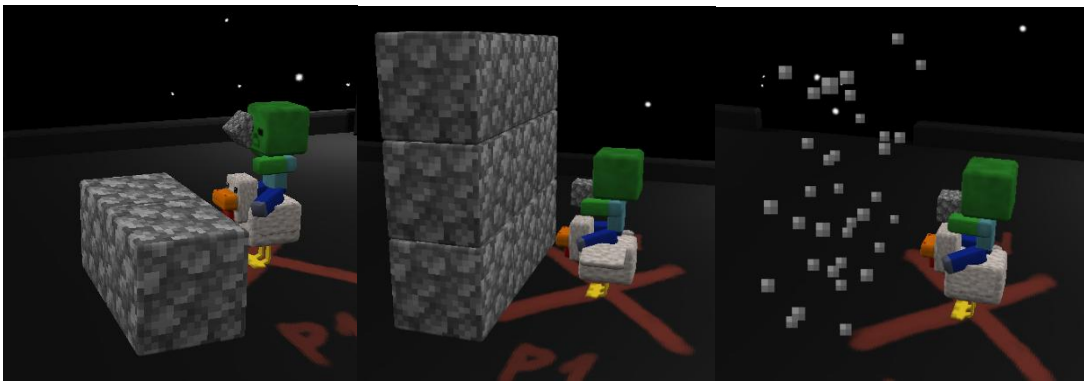
Attack Animation

The baby zombie will hold an egg. This is done by drawing an egg particle at the end of its right arm. He will then start shaking his right arm, resembling a throwing motion. Particle instances of eggs will be spawned every 0.1 seconds and given a velocity towards the opponent and downwards acceleration (gravity), with a slight velocity change in the Y and Z axis so that the eggs can spread out.

When the eggs reach X > 8 (reached opponent), the egg particle will be deleted, and 4 egg shards particle will be spawned in its place, mimicking a break effect. The egg shard particles will be

initiated with a random velocity in the X and Z axis with downwards acceleration. When the egg shard particle reaches Y < 0 (touching the ground), it will be deleted.

Defend Animation



A cobblestone block model will be drawn at the end of the baby zombie's right arm. The arm swinging motion is the same as the attack animation, resembling the "placing blocks" motion in Minecraft.

A total of 9 blocks will be drawn in front of the chicken jockey one by one. The duration between each block "placed" is 0.1 seconds. This is done by using a timer. When the timer reaches the duration, it will reset and include the drawing of the next block.

When animtime > 3 (3 seconds passed since animation played), the blocks will be dissapear and leaves behind particles that resemble shards. The particles are like the egg shards particle, but with a tweak of values in initial velocity.

Regeneration Animation

A potion will be drawn as a flat particle image above the baby zombie's head. The potion is given upwards velocity and downwards acceleration (gravity). When the potion's Y position <

1.5 (reaches baby zombie's head), it will disappear, and 6 animated effect particles will be spawned in its place.

The 8 animated effects are animated using 8 different textures and a timer in each particle object. The particle is first given a random velocity and a upwards acceleration (so that it floats upwards). When the effect particle is spawned, it will start its timer. If the timer reaches the animation duration, it will then change the particle's texture to the next animation frame. When it cycles all the animation frames, it will delete itself.

Victory Animation

The baby zombie's body will rotate –15 to 15 degrees with a sin function in the x axis. His hands will also rotate up and down to resemble a dancing motion. His head is rotated 20 upwards in the Z axis so that he looks at the camera.

Defeat Animation

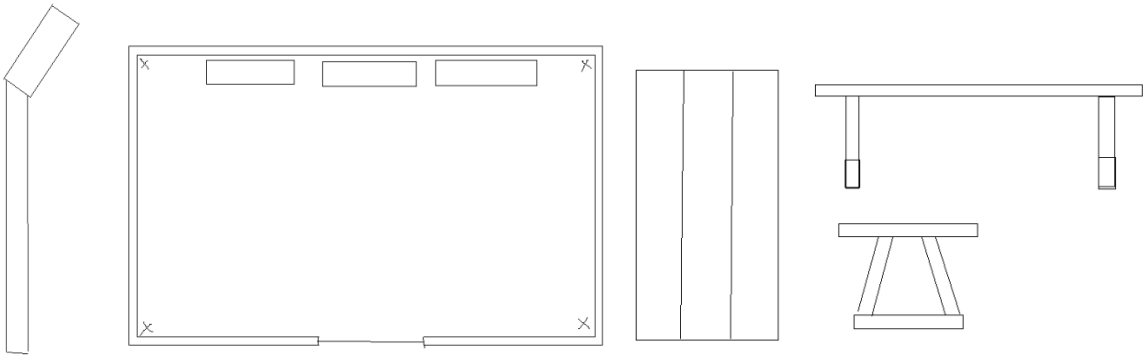
The baby zombie's hands and head are rotated 30 degrees and 40 degrees downwards respectively. The baby's head is also rotated in the Y-axis using a sin function to look like he is shaking his head.

Virtual Environment

The Skybox

We have implemented a technique called Skybox to render a virtual sky in our world. Commonly, skyboxes are made with a cube, sphere or hemisphere. In our case, we are using a sphere. Our skybox is textured with a black sky with white stars. The skybox is turning slowly, to add detail to the environment.

Court Model

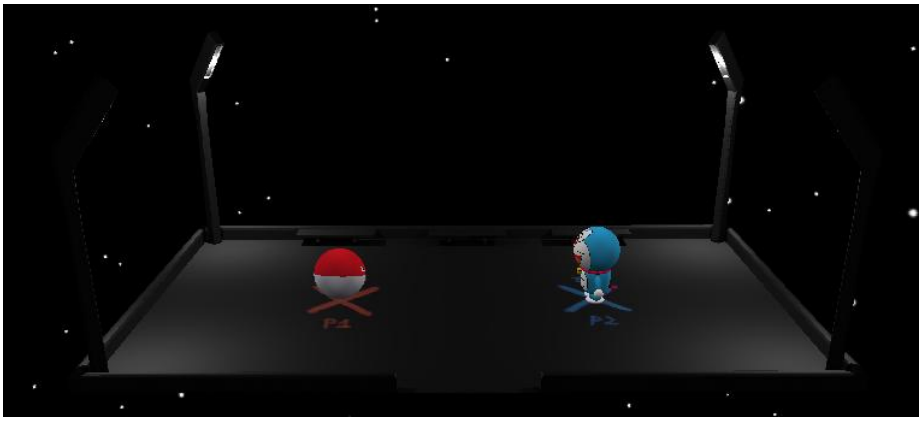


The court model consists of four major parts: the floor, perimeter, bench and light poles.

The floor is just a simple plane with dimensions of 20 x 7.5. The perimeters are resized rectangle cubes joined together. The perimeters top edges are then smoothed.

The light poles are two rectanges cubes joined together. Then a rectange is cut and moved inside for the light. The pole is then resized at the base, neck and top to make it look nicer. Lastly, the model is smoothed.

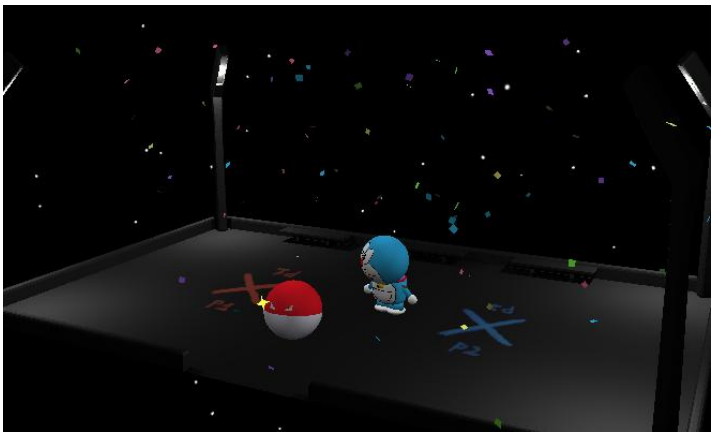
The bench’s top is made of three plank-like flat rectangle cubes. The three planks’ edges are smooth. Then, two legs are added to the bottom of the planks, each leg is made of three rectangular cubes. The three rectangular cubes are joined together and then smoothed.



The court is the place where the characters will fight. The court has four light poles and three benches, along with a low perimeter around the edges.

We have used a technique called Light Baking. Using this technique, we can pre-calculate realistic lighting and shadows as textures and apply it onto the court model. This removes the computational cost to render dynamic shadows to the virtual scene. The downside is that the shadows cannot change in real time, but that's acceptable in our case.

Confetti



The confetti are drawn using GL_QUADS with side of size 0.16. Both the front side and back side are drawn because of GL_CULL. Each confetti particle is an object. The confetti objects are instantiated with a random downward velocity and random velocity in the X and Z axis. It also has a random rotation speed. When the confetti particle touches the ground (y < 0), it will be deleted.

The confetti consist of these five RGB colors:

Color	Hex	RGB
	#a864fd	(168,100,253)
	#29cdff	(41,205,255)
	#78ff44	(120,255,68)
	#ff718d	(255,113,141)
	#fdff6a	(253,255,106)

The confetti only appears when someone wins and doesn't appear when the battle draws.

Graphical User Interface (GUI)

The Graphical User Interface mainly consists of 2d bitmap text fonts (drawn using glutBitmapCharacter) and 2d textures.

The 2d GUI are drawn in a 2d perspective after all 3d elements are drawn, this is to ensure all GUI appear on top of other contents.

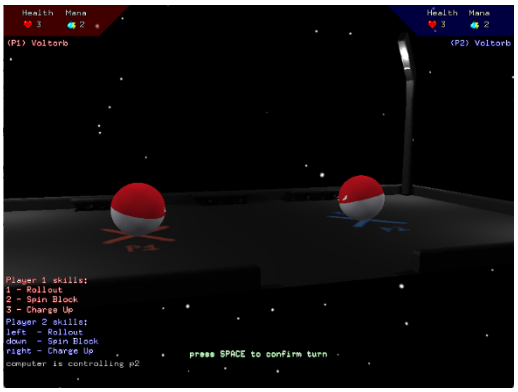
Menu Screen

The “GLUT RUMBLE” logo is a 2d texture drawn to the screen, it bobs up and down and changes its size slightly to give it a more dynamic feel.

There are two text lines in the bottom middle of the screen with instructions. These text lines have a slow blinking brightness to draw attention.

There are four lines of text at the bottom left of the screen to indicate the chosen characters. With colors to differentiate the players.

Battle Screen



There are four main elements in the battle screen: player statistics, skill descriptions, game information and battle instruction.

These elements consist mostly of 2d bitmap text. The information regarding each player is colored differently.

The “press SPACE to confirm turn” text also blinks slowly.

The player statistics area has a transparent background that is drawn with four vector coordinates with transparency of 0.5. The names of the characters are also shown. Two image textures representing health and mana are also drawn at the left side of their corresponding values.

The “glowing effect” or colored border of the bitmap text is done by drawing 4 other bitmap texts offset by 1 pixel in all four directions (up, down, left, right), then drawing the middle text.

Result Screen

The “P1 WIN”, “P2 WIN” and “DRAW” are all 2d textures made by our team.

There is a 2d bitmap text at the bottom of the screen for instructions. It blinks slowly.

USER MANUAL AND INSTRUCTIONS

General Gameplayh

The game is a Pokémon-like turn-based battle game, where at the start of the game, two players will choose their characters. No characters give you an advantage in any way. All characters play the same, only with different skill animations.

After choosing the characters, the battle will begin. In the battle stage, players will choose their skills for the current turn. By confirming the turn, the skills will be cast. There are three types of skills and here's how the skills work:

- Attack: consumes 1 mana, decreases opponent health by 0 if opponent defends, 1 if opponent attacks, 2 if opponent regenerates.
- Defend: consumes no mana, nullifies attacks.
- Regenerate: consumes no mana, will gain 1 mana no matter what.

Here's all possible interactions:

- P1 Attack vs P2 Attack = P1 and P2 both lose 1 health 1 mana.
- P1 Attack vs P2 Defend = P1 loses 1 mana, P2 no change.
- P1 Attack vs P2 Regenerate = P1 loses 1 mana, P2 loses 2 health and gains 1 mana.
- P1 Defend vs P2 Defend = P1 and P2 no change.
- P1 Defend vs P2 Regenerate = P1 no change, P2 gains 1 mana.
- P1 Regenerate vs P2 Regenerate = P1 and P2 both gain 1 mana.

Every character has special names for each skill.

Players' health cannot be increased. The battle will go on until one or both players' health reaches 0. When that happens, the game will enter the result screen.

The game has an option to replace P2 with a computer, which means P2 can't be controlled, and its skills are chosen using a simple algorithm by the computer. This option is ENABLED by default.

The program has some useful features like changing the animation/game speed, locking health and mana (godmode), free camera movement (freeview, noclip), and more.

Menu Screen

In the menu, users can use the button "n" or "m" to select player 1 and player 2 characters respectively. Please note that Caps lock will affect the input, so be careful.

After the user is satisfied with the selection, users can press "space" button to enter the battle screen/stage.

Battle Screen

In the battle screen, users have to choose the skill that will be cast by the characters.

Player 1 can choose skills using key “1”, “2” and “3”.

Player 2 can choose skills using key “left”, “up” and “right”.

The three keys used by players are respectively for “Attack”, “Defend” and “Regenerate”.

By default, the computer will be controlling player 2, so pressing “left”, “right” and “up” keys will not do anything, please be aware of this.

You can press the “I” key to toggle computer control P2.

The last key pressed before confirming the turn will be the skill chosen. The skill will not be displayed and is only known by the player controlling. This is to prevent players from seeing another player’s skill choice. Instead, the skills chosen in the previous turn will be logged into the console.

Pressing the “space” key during battle will confirm the current turn.

After confirming the turn, the game will play the characters’ animations and hide the GUI for 3 seconds, then the GUI will appear again.

Result Screen

In the result screen, you can press the “space” button to return to the menu screen.

Other Controls

Pressing the “f” key will toggle the freeview/noclip mode, this allows the user to freely move the camera around the world.

In freeview/noclip mode, use “wasd” to move around in the horizontal plane, use “q” or “e” to move vertically, use “z” or “x” to zoom in or out, use mouse drag click to look around.

When freeview mode is activated, the camera position information will be displayed at the bottom left corner.

Pressing the “r” key will toggle a visible axis.

Pressing the “o” key will mute or unmute the music.

Pressing the “p” key will set the game speed to 0 is game speed > 0, else set to 1.

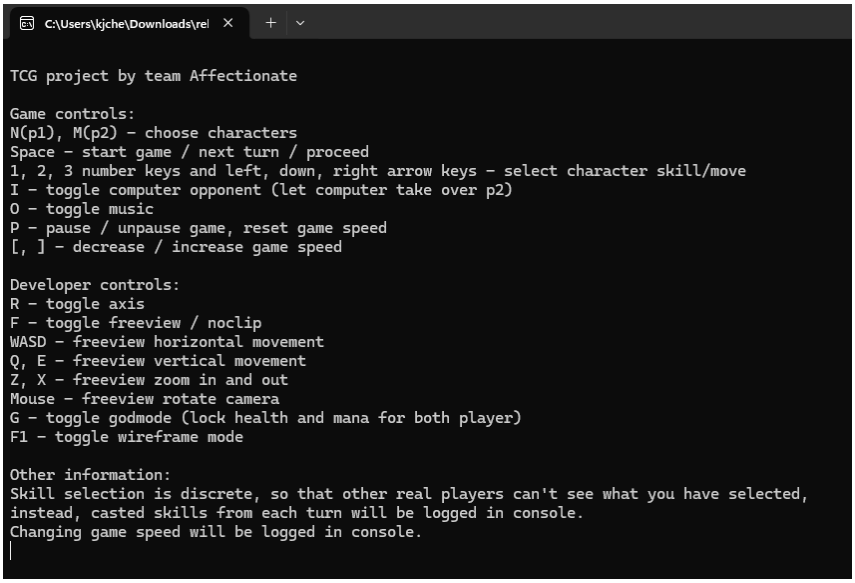
Pressing the “[” or “]” key will slow down or speed up the game speed.

Pressing the “g” key during the battle will lock or unlock both players’ health and mana.

Pressing the “F1” key will toggle wireframe rendering mode.

SCREENSHOTS AND SHOWCASES

Console at Startup



Menu Screen

Initially

It looks like a court somewhere in outer space. Two Voltorbs are at a standoff.

Changed Characters

The characters have been changed from Voltorbs to Chicken Jockey and Doraemon.

Battle Screen

Battle started

The players' statistics and skill descriptions are displayed. It's Voltorb vs Doraemon.

Characters fighting

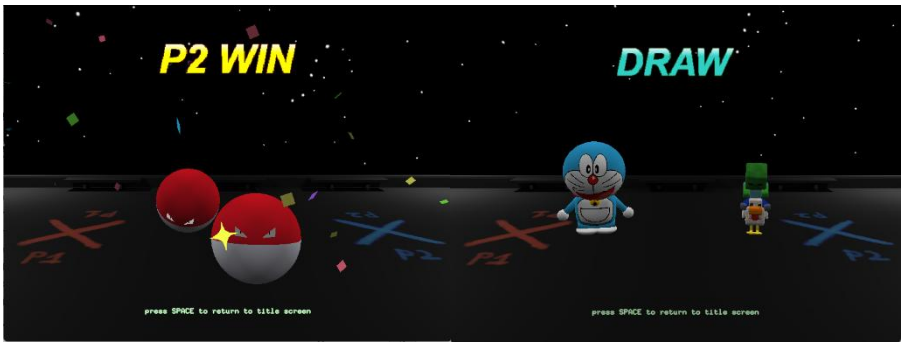
Both characters chose to attack. Using freeview (noclip) mode here to capture at a better angle.

<u>Result Screen</u>

Player 1 wins

Voltorb defeated Doraemon.

Others



Other combinations in the result screen.

DEMONSTRATION VIDEO LINK

Link: https://youtu.be/oG4Vv6yjQTw